

Wazuh Detection Rules

Study Notes — SOC Homelab
Jimmy Mwendwa | Nairobi, Kenya | 2026

These notes document the concepts, structure and examples learned while building custom Wazuh detection rules for a SOC homelab environment. Rules are mapped to MITRE ATT&CK; techniques.

1. How Wazuh Processes Events

Before writing rules, you must understand the pipeline Wazuh follows every time something happens on a monitored endpoint:

Step	What Happens	Example
1	Something happens on server	SSH login fails
2	OS writes a log entry	/var/log/auth.log updated
3	Agent reads log, sends to manager	Agent forwards raw log
4	Manager runs it through a DECODER	Extracts: user, IP, port
5	Manager checks against RULES	Does this match a pattern?
6	If matched → ALERT fires	Alert appears in dashboard

KEY INSIGHT: Rules do not read raw log text. They read FIELDS that decoders already extracted. This is why decoders run before rules in the pipeline.

2. Anatomy of a Wazuh Rule

Here is the simplest possible rule, with every part explained:

```
<rule id="100001" level="5">
  <if_sid>5716</if_sid>
  <description>SSH login failed</description>
</rule>
```

Element	Meaning	Notes
id="100001"	Unique rule number	Custom rules MUST be 100000+ Below that is reserved for Wazuh
level="5"	Severity 0-16	3=low, 7=medium, 10=high, 12+=critical
if_sid	Fire only if this rule ID already fired	You can chain onto existing rules
description	What shows in the dashboard	Keep it clear and specific

3. if_sid vs if_matched_sid

This is one of the most important concepts in Wazuh rule writing:

	if_sid	if_matched_sid
What it asks	Did rule X fire ONCE?	Did rule X fire MULTIPLE times?
Used for	Simple chaining (single event)	Frequency/counting rules (repeated events)
Example	If login succeeded (one event)	If login failed 8 times (counting events)
Rule example	Rule 5760 uses if_sid 5700 (any sshd message)	Rule 5763 uses if_matched_sid 5760 (counts failed passwords)

4. Frequency Rules (The Most Useful Pattern)

Frequency rules detect when something happens X times in Y seconds. This covers 80% of useful security detections:

```
<rule id="100002" level="10" frequency="5" timeframe="60">
  <if_matched_sid>5760</if_matched_sid>
  <same_source_ip/>
  <description>SSH brute force - 5 failed attempts in 60s</description>
</rule>
```

Attribute	Value	Meaning
frequency	5	Must fire this many times...
timeframe	60	...within this many seconds
if_matched_sid	5760	Counting THIS rule's matches
same_source_ip	(no value)	Only count from the SAME attacker IP
level	10	Alert severity when threshold hit

WHY same_source_ip MATTERS: Without it, 5 different people each failing once would trigger the rule. With it, only 1 person failing 5 times triggers it — which is the actual attack pattern you want to catch.

5. Real Example: The SSH Rule Chain

This is what we discovered by reading Wazuh's built-in SSH rules file (0095-sshd_rules.xml). Every SSH log flows through this chain:

Rule ID	Level	Trigger	Description
5700	0 (no alert)	decoded_as: sshd	Parent — catches ALL sshd messages. noalert=1 means never shows in dashboard. Just groups messages together.
5710	5	if_sid: 5700 match: illegal user	Non-existent user tried to login. Child of 5700.
5716	5	if_sid: 5700 match: ^Failed	Single failed authentication. Any failure starting with 'Failed'.
5760	5	if_sid: 5700,5716 match: Failed password	More specific — failed password. Builds on both 5700 AND 5716.
5763	10	if_matched_sid: 5760 frequency: 8 / 120s same_source_ip	BRUTE FORCE detected. Wazuh's built-in brute force rule. Fires after 8 failures in 120s.
5715	3	if_sid: 5700 match: ^Accepted	Successful SSH login. Any 'Accepted' in sshd log.

LESSON: Always check what Wazuh already detects before writing a new rule. Wazuh already has brute force detection (5763). Our custom rules should either lower the threshold, add context, or detect things Wazuh doesn't cover.

6. Our Custom Rules (local_rules.xml)

Rule 100001 — Successful Login After Brute Force (CRITICAL)

```
<rule id="100001" level="14">
  <if_matched_sid>5763</if_matched_sid>
  <if_sid>5715</if_sid>
  <same_source_ip/>
  <description>CRITICAL: Successful SSH login after brute force</description>
  <mitre><id>T1110</id><id>T1078</id></mitre>
</rule>
```

Element	Why We Used It
if_matched_sid: 5763	5763 is the brute force rule. if_matched_sid means we need it to have COUNTED enough failures.
if_sid: 5715	5715 fires on successful login. We need BOTH: brute force happened AND then success.
same_source_ip	The success must come from the SAME IP that was brute forcing.
level: 14	Critical — attacker is now INSIDE the network with valid credentials.
T1078	Valid Accounts — attacker used legitimate credentials (possibly obtained via brute force T1110).

Rule 100002 — Stricter Brute Force Threshold

```
<rule id="100002" level="10" frequency="5" timeframe="60">
  <if_matched_sid>5760</if_matched_sid>
```

```
<same_source_ip/>
<description>SSH brute force - 5 failed attempts in 60s</description>
<mitre><id>T1110</id></mitre>
</rule>
```

Element	Why We Used It
frequency: 5 / timeframe: 60	Stricter than Wazuh default (8/120s). In your homelab, ANY 5 failures in 60s is suspicious —
if_matched_sid: 5760	5760 catches 'Failed password'. We count those specifically.
same_source_ip	Only count failures from the SAME attacker IP address.

7. MITRE ATT&CK; Techniques Used

Technique	Name	What It Means	In Your Rules
T1110	Brute Force	Attacker repeatedly tries passwords to gain access	Rules 100001, 100002 Wazuh built-in: 5763
T1110.001	Password Guessing	Specific variant: guessing passwords for known accounts	Rule 5760 (built-in)
T1078	Valid Accounts	Attacker uses legitimate credentials to access system	Rule 100001, 5715
T1021.004	SSH Remote Services	Using SSH specifically for remote access	Rule 5760 (built-in)
T1046	Network Service Discovery	Port scanning to find open services	Future rule for nmap
T1136	Create Account	Attacker creates new account for persistence	Future rule for useradd
T1548	Abuse Elevation Control	Using sudo to gain root privileges	Future rule for sudo

8. Wazuh Alert Severity Levels

Level	Severity	Action	Example
0-3	Informational	No action needed	System startup, normal logins
4-6	Low	Monitor	Single failed login, config change
7-9	Medium	Investigate when time allows	Multiple failures, policy violation
10-11	High	Investigate soon	Brute force detected, new admin account
12-13	Critical	Investigate immediately	Root login, malware detected
14-15	Emergency	Stop everything, investigate now	Denial of service confirmed, ransomware

9. How to Find Rule IDs to Chain From

Before writing any custom rule, always search Wazuh's built-in rules first:

```
# Find rules matching a log pattern
sudo grep -rn 'Failed password' /var/ossec/ruleset/rules/

# List all rule IDs in an SSH rules file
sudo grep -n 'rule id' /var/ossec/ruleset/rules/0095-sshd_rules.xml

# View a specific rule by line number
sudo sed -n '148,157p' /var/ossec/ruleset/rules/0095-sshd_rules.xml

# Find the SSH rules file
sudo find /var/ossec -name '*sshd*'
```

RULE: Always check what Wazuh already detects before writing a new rule. Writing a duplicate rule wastes time and can cause conflicts. Build ON TOP of existing rules using `if_sid` or `if_matched_sid`.

10. Rule Writing Checklist

Step	Question to Ask	Why It Matters
1	Does Wazuh already detect this?	Don't duplicate existing rules. Search ruleset first.
2	What existing rule fires first?	You need the parent rule ID to chain from (if_sid).
3	Single event or repeated event?	Single → if_sid Repeated → if_matched_sid + frequency
4	Same attacker or any source?	Brute force → same_source_ip General detection → omit it
5	What level is appropriate?	Match severity to actual risk. Don't cry wolf with level 14.
6	What MITRE technique applies?	Check attack.mitre.org Maps your rule to real attacks.
7	Is the description clear?	Someone else should understand the alert without context.
8	Validate syntax before restart	<code>sudo /var/ossec/bin/wazuh-logtest</code> Catch typos before they crash manager.

11. Key Commands Reference

Command	Purpose
<code>sudo nano /var/ossec/etc/rules/local_rules.conf</code>	Edit your custom rules file
<code>sudo /var/ossec/bin/wazuh-logtest</code>	Validate rules syntax and test log matching
<code>sudo systemctl restart wazuh-manager</code>	Apply rule changes
<code>sudo systemctl status wazuh-manager</code>	Check manager started successfully
<code>sudo tail -f /var/ossec/logs/alerts/alerts.json</code>	Watch alerts in real time
<code>sudo tail -f /var/ossec/logs/ossec.log</code>	Watch manager logs for errors
<code>sudo grep -rn 'pattern' /var/ossec/ruleset</code>	Search built-in rules for a pattern
<code>sudo grep -n 'rule id' /var/ossec/ruleset</code>	List all rule IDs in a file
<code>sudo sed -n '148,157p' /var/ossec/ruleset</code>	View specific lines of a rules file

12. Key Lessons Learned

1. Rules don't read raw logs — they read fields that decoders already extracted.
2. Custom rules MUST use IDs 100000 and above. Below that is reserved for Wazuh.
3. Always search existing rules before writing a new one — Wazuh already detects most common attacks.
4. if_sid = chain from a single event. if_matched_sid = count repeated events.
5. same_source_ip is critical for brute force rules — without it you count different people, not one attacker.

6. Stricter thresholds make sense in your homelab because there are no legitimate remote users.
7. A typo in the description tag (like) will crash the entire rule set — always validate.
8. MITRE ATT&CK; IDs in rules connect your detection to real-world attack patterns and help with reporting.
9. Level 14 should be reserved for confirmed or near-confirmed compromises — don't overuse high levels.
10. The wazuh-logtest tool lets you paste a raw log line and see exactly which rules would fire.

These notes are part of the SOC Homelab project by Jimmy Mwendwa | GitHub:
github.com/munchydevante-bot/SOC-Homelab